AI
Agents

Module - 3

Agent Memory

-

muhammadosama.hq@gmail.com

Agent Memories

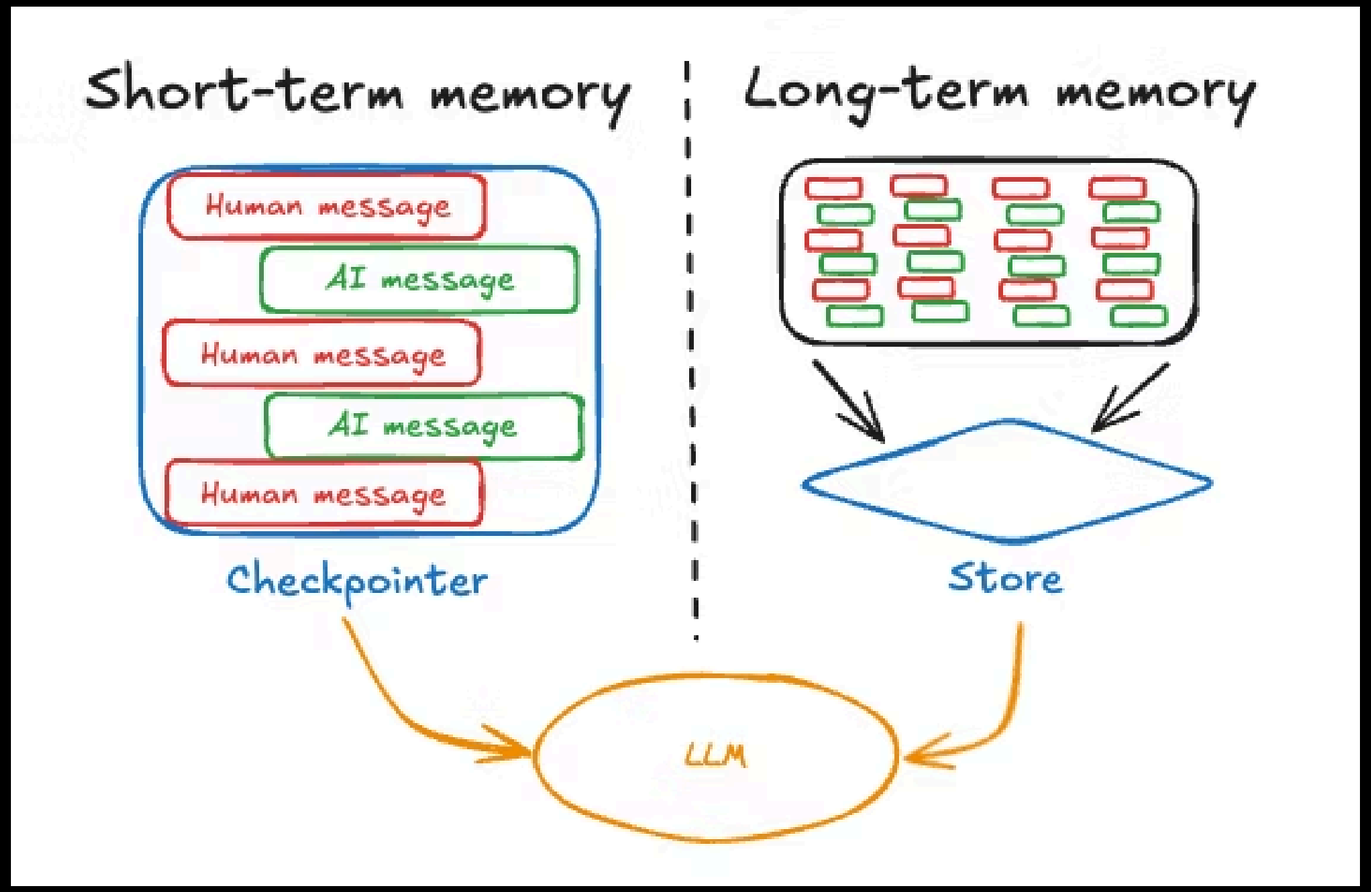
Memory is a system that remembers information about previous interactions. For AI agents, memory is crucial because it lets them remember previous interactions, learn from feedback, and adapt to user preferences.

Two Kind of Memories

AI applications need memory to share context across multiple interactions. In LangGraph, you can add two types of memory:

- Short-term memory is a part of your agent's state to enable multi-turn conversations.
- Long-term memory to store user-specific or application-level data across sessions.

Two Kind of Memories



Short-Term Memory

-

Short-Term Memory

Short-term memory (thread-level persistence) enables agents to track multi-turn conversations.

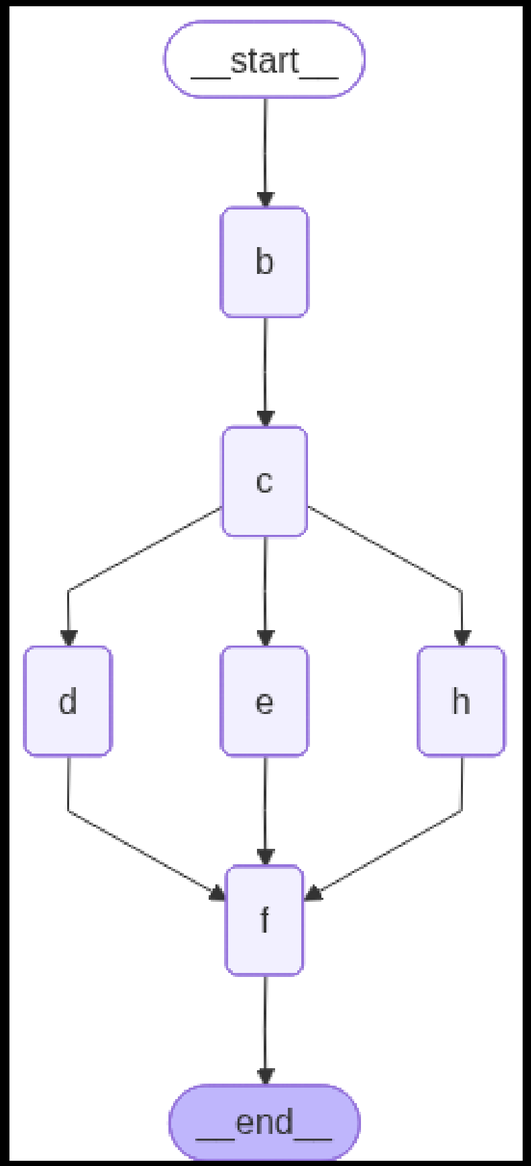
In Langgraph, short-term memory is implemented by persistance

Persistance

-

What is Persistance?

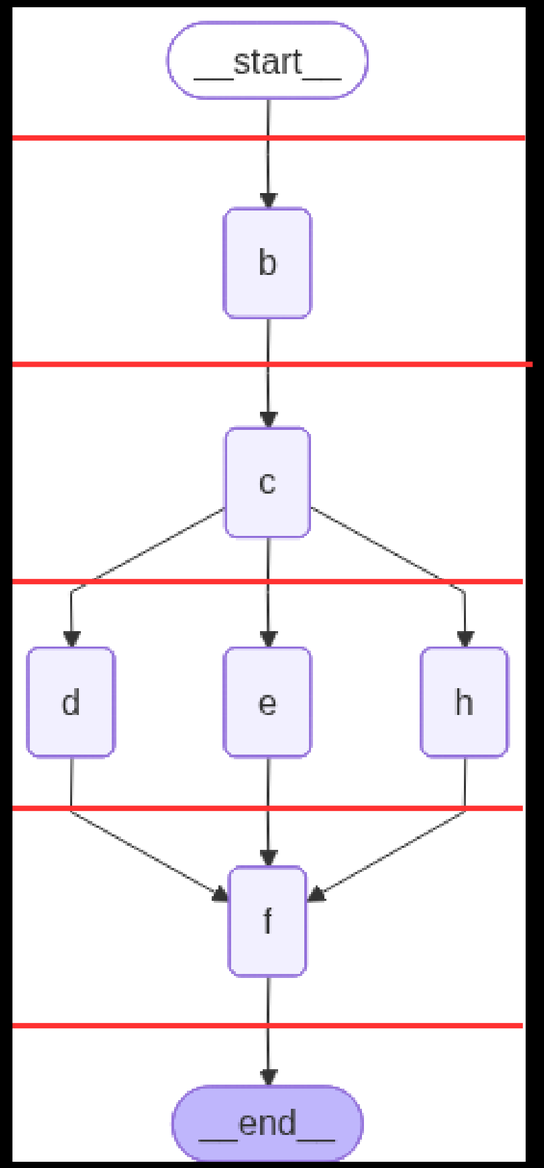
Persistence in LangGraph, It means automatically saving your graph's state after each step (Super Step)



Checkpoints

What is Checkpointer?

Checkpointer is a concept through persistance is implemented. It divides the whole graph into checkpoints.



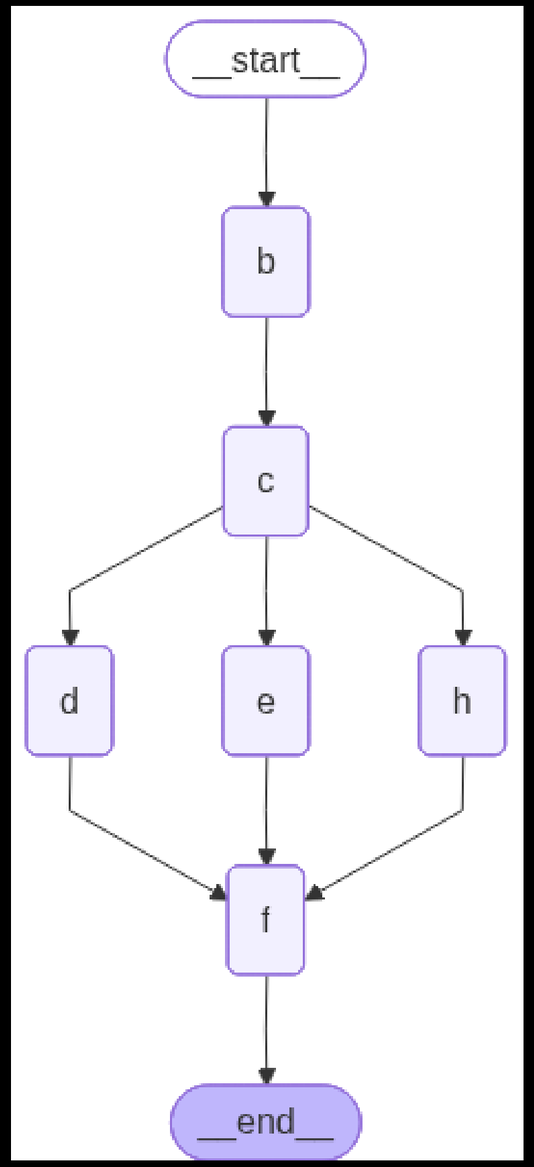
Checkpoint is a snapshot of the graph state saved at each super-step

AI
Agents

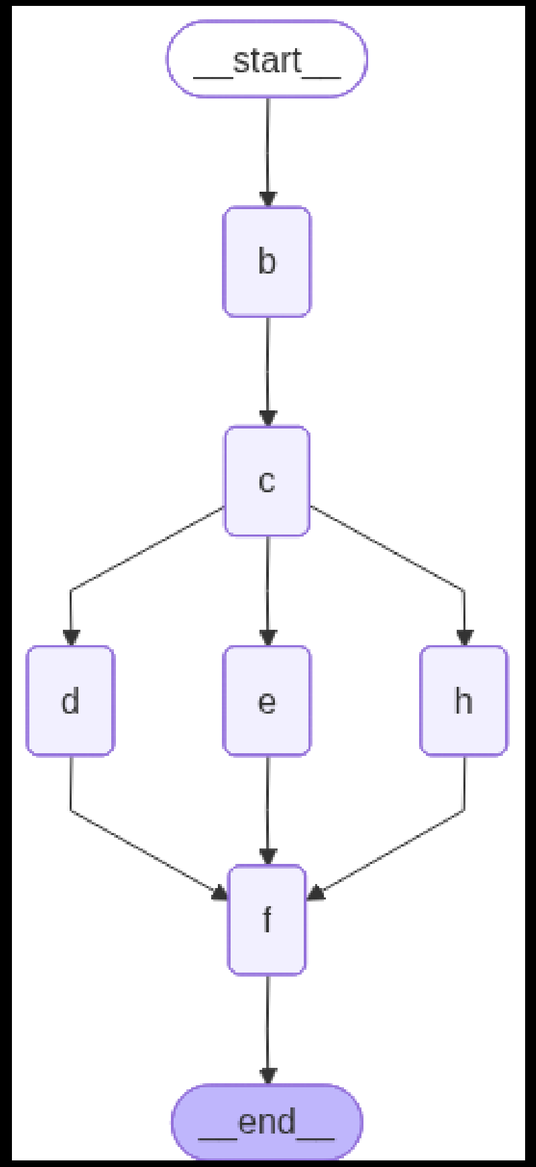
What is Thread?

A thread is a unique ID or thread identifier assigned to each checkpoint saved by a checkpointer.

Thread Id 1



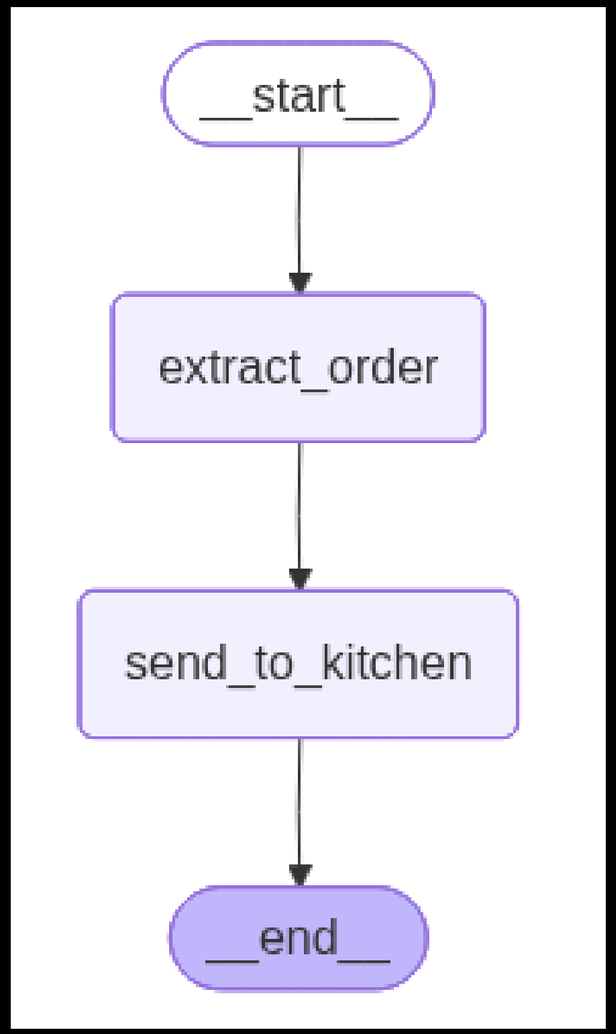
Thread Id 2



Implement Persistance

AI Restaurant Order Processing Bot

The purpose of this bot is to understand a **customer's food order written in natural language**, and simulate sending that order to the kitchen just like what happens in real restaurants, and food delivery apps.



AI Agents

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv

load_dotenv()

class OrderState(TypedDict):
    order_id: str
    user_message: str
    items: list[str]

llm = ChatGoogleGenerativeAI(
    model="gemini-3-flash-preview",
    temperature=0
)
```

AI Agents

```python
def extract_order(state: OrderState) -> OrderState:
    prompt = f"""
    Extract all food items from the customer's message.
    Return items as a comma-separated list.
    If no food items are found, return NONE.

    Message: "{state['user_message']}"
    """

    response = llm.invoke(prompt).content

    items = state.get("items", [])
    r = response[0].get("text", "None")

    if r != "NONE":
        extracted_items = [item.strip() for item in r.split(",")]
        items.extend(extracted_items)
    return {
        "items": items
    }
```

AI Agents

```python
def send_to_kitchen(state: OrderState) -> OrderState:
    items_formatted = "\n".join(f"- {item}" for item in state["items"])

    # ---- Simulating kitchen processing ----
    print("\n👩‍🍳 Sending order to kitchen...")
    print(f"Order ID: {state['order_id']}")
    print("Items:")
    print(items_formatted)

    # ---- Customer-facing message ----
    prompt = f"""
You are a polite restaurant assistant.

Inform the customer that their order has been placed successfully.
Mention the ordered items and say it will take 20 to 25 minutes to serve.
Thank them for their patience.

Ordered items:
{items_formatted}
"""
    response = llm.invoke(prompt).content
    print("\nAI:", response[0].get("text", None))
```

AI
Agents

```
workflow = StateGraph(OrderState)

workflow.add_node("extract_order", extract_order)
workflow.add_node("send_to_kitchen", send_to_kitchen)

workflow.add_edge(START, "extract_order")
workflow.add_edge("extract_order", "send_to_kitchen")
workflow.add_edge("send_to_kitchen", END)

checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)
```

Call the Agent → Customer 1

```
graph.invoke(
    {
        "order_id": order_id,
        "user_message": "I would like a large pizza and a cold drink",
        "items": []
    },
    config={"thread_id": order_id}
)
```

Call the Agent → Customer 2

```
order_id = "ORDER-89"

graph.invoke(
    {
        "order_id": order_id,
        "user_message": "please take order of 1 kg karhai, 4 naan and 1 liter cold drink",
        "items": []
    },
    config={"thread_id": order_id}
)
```

Benifits of Persistance

- Memory
- Fault-tolerance
- Time Travel
- Human-in-the-loop

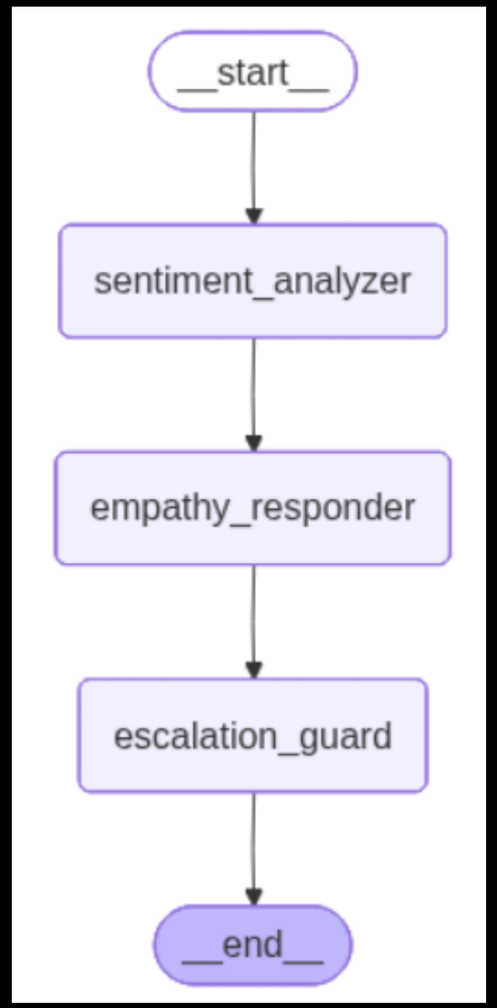
Use Case

Intelligent Customer Support Automation

In e-commerce, telecom, banking, and SaaS, AI Support bot is a must requirement in today's world to provide support to large customer base in an efficient way.

Workflow Nodes

- Sentiment Analyzer: Classifies the emotional tone of the user message and updates the sentiment score and user message in state.
- Empathy Responder: Sends full conversation history to the LLM and generates a tone adjusted reply.
- Escalation Guard: Checks sentiment and turn count, then routes to Human or Agent



Code Walkthrough

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv
from langchain_core.messages import HumanMessage, AIMessage, BaseMessage
from langgraph.graph.message import add_messages
from pydantic import BaseModel, Field

load_dotenv()
```

Code Walkthrough

```
class SupportState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    user_input: str
    sentiment_score: float
    turn_count: int
    reply: str

class SentimentScore(BaseModel):
    score: float = Field(
        description="Sentiment score between 0.0 (very angry) and 1.0 (very happy)",
        ge=0.0,
        le=1.0
    )
```

Code Walkthrough

```python
llm = ChatGroq(model="llama-3.1-8b-instant", temperature=0)

def sentiment_analyzer(state: SupportState) -> SupportState:
    structured_llm = llm.with_structured_output(SentimentScore)

    prompt = PromptTemplate(
        template='''Classify the sentiment of this customer message.
            Return a score between 0.0 (very angry) and 1.0 (very happy).
            Message: {user_message}''',

        input_variables=["user_message"]
    )
    chain = prompt | structured_llm
    result: SentimentScore = chain.invoke({"user_message": state["user_input"]})

    return {
        "messages": [HumanMessage(content=state["user_input"])],
        "sentiment_score": result.score
    }
```

Code Walkthrough

```python
def empathy_responder(state: SupportState) -> SupportState:
    score = state["sentiment_score"]
    if score < 0.3:
        tone = "The customer is frustrated. Be apologetic and focus on de-escalation."
    elif score < 0.6:
        tone = "The customer is somewhat unhappy. Be warm and solution-focused."
    else:
        tone = "The customer is calm. Be friendly and efficient."

    prompt = ChatPromptTemplate([
        ("system", "You are a customer support agent. {tone} Keep replies to 2-3 sentences."),
        ("placeholder", "{conversation}"),
        ("human", "{question}")
    ])

    chain = prompt | llm | StrOutputParser()
    reply = chain.invoke({
        "tone": tone,
        "conversation": state["messages"],
        "question": state["user_input"]
    })
    return {"messages": [AIMessage(content=reply)], "reply": reply}
```

Code Walkthrough

```python
def escalation_guard(state: SupportState) -> SupportState:
    new_count = state.get("turn_count", 0) + 1

    if state["sentiment_score"] < 0.25 or new_count >= 6:
        return {
            "turn_count": new_count,
            "reply": "Your query has been forwarded to a human agent. Thank you for reaching out."
        }

    return {"turn_count": new_count}
```

Code Walkthrough

```
graph = StateGraph(SupportState)
graph.add_node("sentiment_analyzer", sentiment_analyzer)
graph.add_node("empathy_responder", empathy_responder)
graph.add_node("escalation_guard", escalation_guard)

graph.add_edge(START, "sentiment_analyzer")
graph.add_edge("sentiment_analyzer", "empathy_responder")
graph.add_edge("empathy_responder", "escalation_guard")
graph.add_edge("escalation_guard", END)

workflow = graph.compile()
```

Code Walkthrough

```python
print("Support Agent — type 'quit' to exit\n")
while True:
    user_input = input("You: ").strip()
    if user_input.lower() == "quit":
        break

    execution_response = workflow.invoke({
        "user_input": user_input,
    })

    print(f"\nAgent: {execution_response['reply']}")
    print(f"[sentiment: {execution_response['sentiment_score']} | turn: {execution_response["turn_count"]}]\n")
    print(f"Conversation Messages: \n{execution_response["messages"]}")
```

Short-Term Memory

To add short term memory in our usecase add following code:

Necessary Import

```
from langgraph.checkpoint.memory import InMemorySaver
```

Add Checkpointer to Workflow

```
checkpointer = InMemorySaver()
workflow = graph.compile(checkpointer=checkpointer)
```

Short-Term Memory

Call Workflow with Config

```
execution_response = workflow.invoke({
        "user_input": user_input,
    },
config={"thread_id": 1}
)
```

Fault-tolerance

Checkpointing provides two kinds of fault tolerance for error recovery: if a node fails at a given step or the server crashes, the graph resumes from the last successful checkpoint.

- Node failure: a retry policy is applied at the node level to re-execute the failed step.
- Server crash: once the server restarts, the workflow resumes automatically from the last saved checkpoint.

Fault-tolerance - Node failure

Add node level retry policy to minimize node failure

Necessary Import

```
from langgraph.types import RetryPolicy
```

Add Retry Policy at Nodes

```
graph.add_node("sentiment_analyzer", sentiment_analyzer, retry=RetryPolicy())
graph.add_node("empathy_responder", empathy_responder, retry=RetryPolicy())
graph.add_node("escalation_guard", escalation_guard, retry=RetryPolicy())
```

Fault-tolerance - Node failure

To Add Custom Retry Policy

```
node_retry = RetryPolicy(
    max_attempts=2,
    initial_interval=0.2,
    retry_on=Exception # Retry on every exception
    )
```

Apply default Policy

```
graph.add_node("sentiment_analyzer", sentiment_analyzer, retry=node_retry)
graph.add_node("empathy_responder", empathy_responder, retry=node_retry)
graph.add_node("escalation_guard", escalation_guard, retry=node_retry)
```

Fault-tolerance - Server crash

To implement server crash prevent add below logic in code:

Necessary Import

```
from langgraph.checkpoint.sqlite import SqliteSaver
```

SQL Lite Checkpointer Setup

```
# SqliteSaver writes every checkpoint to a .db file on disk
# so state survives a server crash or restart
with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
    workflow = graph.compile(checkpointer=checkpointer)

    config = {"configurable": {"thread_id": 1}}

    # Check for any incomplete (crashed) execution on startup
    print("Checking for interrupted sessions...\n")
    state_snapshot = workflow.get_state(config)
```

Fault-tolerance - Server crash

```
if state_snapshot and state_snapshot.next:
    # next contains the nodes that were pending when the crash happened
    print(f"Crashed execution found!")
    print(f"   Resuming from node: {state_snapshot.next}")
    print(f"   Last turn count    : {state_snapshot.values.get('turn_count', 0)}")
    print(f"   Last sentiment      : {state_snapshot.values.get('sentiment_score', 'N/A')

    # Resume by invoking with None — tells LangGraph to continue from checkpoint
    recovered_response = workflow.invoke(None, config=config)
    print(f"Recovered response: {recovered_response['reply']}\n")
else:
    print("No interrupted session found. Starting fresh.\n")

print("State History")
history = list(workflow.get_state_history(config))
```

Time Travel

LangGraph supports time travel through checkpoints with two features written below:

- Replay: Retry from a prior checkpoint.
- Fork: Branch from a prior checkpoint with modified state to explore an alternative path.

Replay - Code Walkthrough

Perform reply technique for debugging

```
from langgraph.graph import StateGraph, START
from langgraph.checkpoint.memory import InMemorySaver
from typing_extensions import TypedDict, NotRequired
from langchain_core.utils.uuid import uuid7
```

Replay - Code Walkthrough

```python
class State(TypedDict):
    topic: NotRequired[str]
    joke: NotRequired[str]

def generate_topic(state: State):
    return {"topic": "socks in the dryer"}

def write_joke(state: State):
    return {"joke": f"Why do {state['topic']} disappear? They elope!"}
```

Replay - Code Walkthrough

```
# Step 1: Run the graph
config = {"configurable": {"thread_id": str(uuid7())}}
result = graph.invoke({}, config)

# Step 2: Find a checkpoint to replay from
history = list(graph.get_state_history(config))
# History is in reverse chronological order
for state in history:
    print(f"next={state.next}, checkpoint_id={state.config['configurable']['checkpoint_id']}")
```

Replay - Code Walkthrough

```
# Step 3: Replay from a specific checkpoint
# Find the checkpoint before write_joke
before_joke = next(s for s in history if s.next == ("write_joke",))
replay_result = graph.invoke(None, before_joke.config)
# write_joke re-executes (runs again), generate_topic does not
```

Replay - Code Walkthrough

```
# Check updated history
history = list(graph.get_state_history(config))
# History is in reverse chronological order
for state in history:
    print(f"next={state.next}, checkpoint_id={state.config['configurable']['checkpoint_id']}")
```

Fork - Code Walkthrough

Add below code in previous reply script

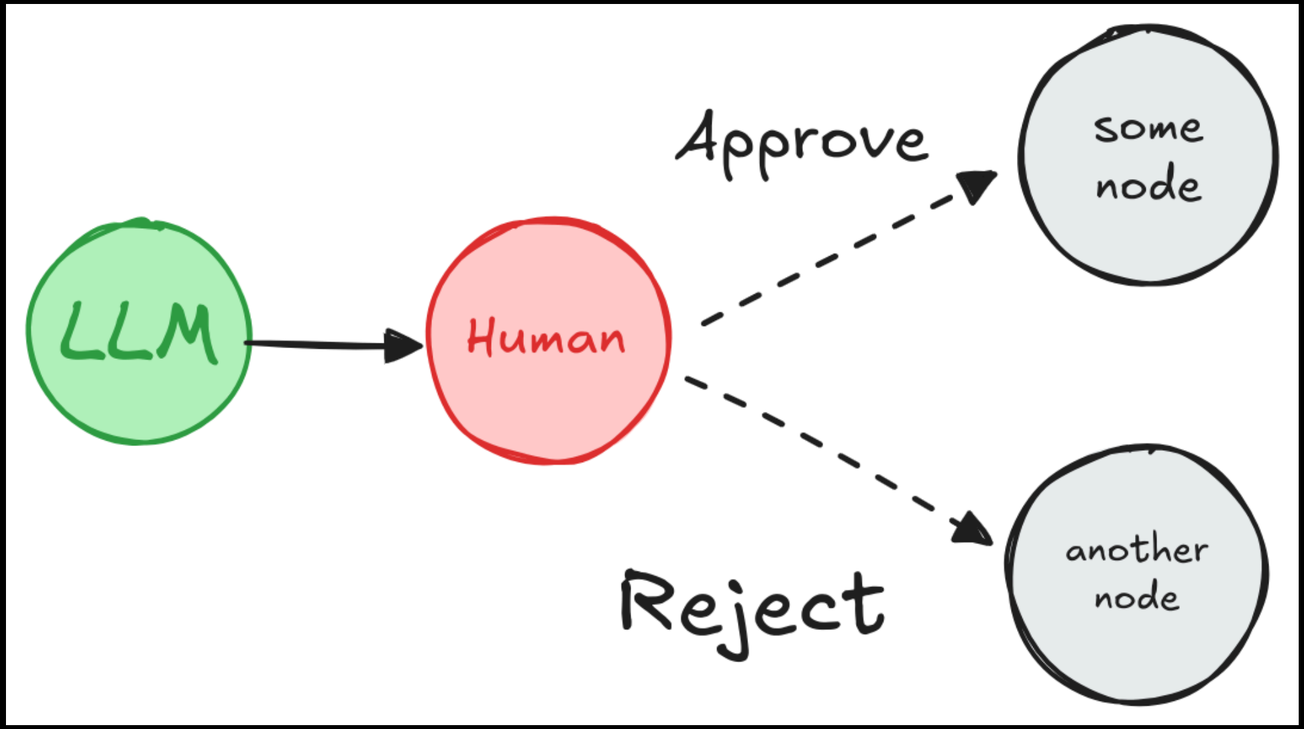
```
# Find checkpoint before write_joke
history = list(graph.get_state_history(config))
before_joke = next(s for s in history if s.next == ("write_joke",))

# Fork: update state to change the topic
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
)

# Resume from the fork — write_joke re-executes with the new topic
fork_result = graph.invoke(None, fork_config)
print(fork_result["joke"])  # A joke about chickens, not socks
```

Human-In-The-Loop (HITL)

Checkpointers facilitate human-in-the-loop workflows by allowing humans to inspect, interrupt, and approve graph steps.

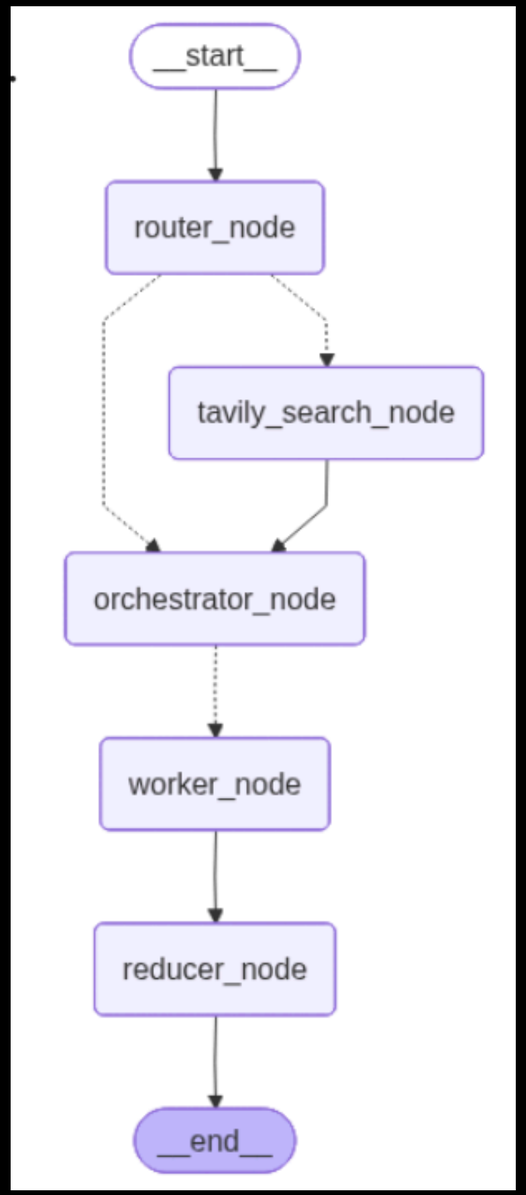


Use Case

AI Tutor

Overview:
Students ask about any query or topic it will generate a comprehensive output to resolve doutes and clear the concept

- User submits a question.
- Router decides whether web research is needed.
- If needed, Tavily fetches and summarizes relevant context.
- Orchestrator breaks the question into 2–5 subtopics.
- Each subtopic is processed by parallel worker nodes.
- Workers generate focused explanations for each subtopic.
- Reducer merges all outputs into a single structured answer.
- Final response is returned to the user.

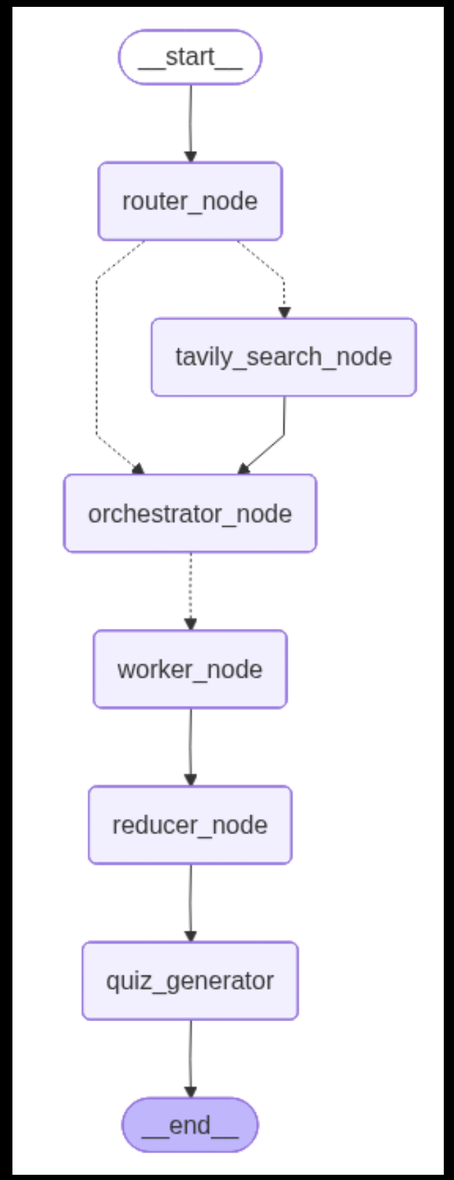


Use Case - HITL Implemented

AI Tutor

Updated Flow

- User submits a question.
- Router decides whether web research is needed.
- If needed, Tavily fetches and summarizes relevant context.
- Orchestrator breaks the question into 2–5 subtopics.
- Each subtopic is processed by parallel worker nodes.
- Workers generate focused explanations for each subtopic.
- Reducer merges all outputs into a single structured answer.
- System pauses and asks the user if they want a quiz (human-in-the-loop approval).
- If approved, a quiz is generated based on the final content.
- Final response (with or without quiz) is returned to the user.



Long-Term Memory

-

Long Term Memory

What if we want to retain some information across threads?

Consider the case of a chatbot, where we want to retain specific information about the user across all chat conversations (e.g., threads) with that user!

Example use case:

- Remembering a user's name or preferences across chats

Purpose:

- Personalizing responses over time

store

-

Long Term Memory → 1

```python
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

namespace1 = ("school", "student", "osama")
namespace2 = ("school", "student", "sara")


# — Save ————————————————————————————————————————————————————————
store.put(namespace1,  "profile", {"name": "osama",  "age": 25, "city": "Karachi", "topics": []})
store.put(namespace2, "profile", {"name": "Sara", "age": 22, "city": "Lahore",  "topics": []})
```

Long Term Memory → 2

```
# ── Load ──────────────────────────────────────────────────────────
osama  = store.get(namespace1,  "profile")
sara = store.get(namespace2, "profile")
print(osama.value)   # {'name': 'osama',  'age': 25, 'city': 'Karachi', 'topics': []}
print(sara.value)  # {'name': 'Sara', 'age': 22, 'city': 'Lahore',  'topics': []}

# ── Update ────────────────────────────────────────────────────────
store.put(namespace1, "profile", {**osama.value, "city": "Dubai", "topics": ["Python", "LangGraph"]})

osama = store.get(namespace1, "profile")
print(osama.value)   # {'name': 'osama', 'age': 25, 'city': 'Dubai', 'topics': ['Python', 'LangGraph']}
```

Long Term Memory → 3

```
# ── List all namespaces ──────────────────────────────────────────────
keys = store.list_namespaces(prefix=("school", "student"))
print(keys)        # [('school', 'student', 'osama'), ('school', 'student', 'sara')]

# ── Delete ───────────────────────────────────────────────────────────
store.delete(namespace2, "profile")
print(store.get(namespace2, "profile"))   # None
```

End of Class

-